

torch.jit.script#

<https://pytorch.org/docs/stable/generated/torch.jit.script.html#torch.jit.sc>

Description:

Scripting a function or `nn.Module` will inspect the source code, compile it as TorchScript code using the TorchScript compiler, and return a `ScriptModule` or `ScriptFunction`. TorchScript itself is a subset of the Python language, so not all features in Python work, but we provide enough functionality to compute on tensors and do control-dependent operations. For a complete guide, see the TorchScript Language Reference. Scripting a dictionary or list copies the data inside it into a TorchScript instance that can be subsequently passed by reference between Python and TorchScript with zero copy overhead. and as a decorator `@torch.jit.script` for torchscript-classes and functions. obj (Callable, class, or `nn.Module`) – The `nn.Module`, function, class type, dictionary, or list to compile.

Function Signature

```
torch.jit.script(obj, optimize=None, _frames_up=0,  
_rcb=None, example_inputs=None)[source]#
```

torch.jit.script can be used as a function for modules, functions, dictionaries and lists

Parameters

Returns

Scripting a function

Returns:

If obj is nn.Module, script returns a ScriptModule object. The returned ScriptModule will have the same set of sub-modules and parameters as the original nn.Module. If obj is a standalone function, a ScriptFunction will be returned. If obj is a dict, then script returns an instance of torch._C.ScriptDict. If obj is a list, then script returns an instance of torch._C.ScriptList.

Code Examples

```
import torch

@torch.jit.script
def foo(x, y):
    if x.max() > y.max():
        r = x
    else:
        r = y
    return r

print(type(foo)) # torch.jit.ScriptFunction

# See the compiled graph as Python code
print(foo.code)

# Call the function using the TorchScript interpreter
foo(torch.ones(2, 2), torch.ones(2, 2))
```

```
import torch

def test_sum(a, b):
    return a + b

# Annotate the arguments to be int
scripted_fn = torch.jit.script(test_sum, example_inputs=[(3,
4)])

print(type(scripted_fn)) # torch.jit.ScriptFunction

# See the compiled graph as Python code
print(scripted_fn.code)

# Call the function using the TorchScript interpreter
scripted_fn(20, 100)
```

```
import torch

class MyModule(torch.nn.Module):
    def __init__(self, N, M):
        super().__init__()
        # This parameter will be copied to the new ScriptModule
        self.weight = torch.nn.Parameter(torch.rand(N, M))

        # When this submodule is used, it will be compiled
        self.linear = torch.nn.Linear(N, M)

    def forward(self, input):
        output = self.weight.mv(input)

        # This calls the `forward` method of the `nn.Linear`
module, which will
        # cause the `self.linear` submodule to be compiled to a
`ScriptModule` here
        output = self.linear(output)
        return output

scripted_module = torch.jit.script(MyModule(2, 3))
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MyModule(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        # torch.jit.trace produces a ScriptModule's conv1 and
conv2
        self.conv1 = torch.jit.trace(nn.Conv2d(1, 20, 5),
torch.rand(1, 1, 16, 16))
        self.conv2 = torch.jit.trace(nn.Conv2d(20, 20, 5),
torch.rand(1, 20, 16, 16))

    def forward(self, input):
        input = F.relu(self.conv1(input))
        input = F.relu(self.conv2(input))
        return input

scripted_module = torch.jit.script(MyModule())
```

```
import torch
import torch.nn as nn

class MyModule(nn.Module):
    def __init__(self) -> None:
        super().__init__()

    @torch.jit.export
    def some_entry_point(self, input):
        return input + 10

    @torch.jit.ignore
    def python_only_fn(self, input):
        # This function won't be compiled, so any
        # Python APIs can be used
        import pdb

        pdb.set_trace()

    def forward(self, input):
        if self.training:
            self.python_only_fn(input)
        return input * 99

scripted_module = torch.jit.script(MyModule())
print(scripted_module.some_entry_point(torch.randn(2, 2)))
print(scripted_module(torch.randn(2, 2)))
```

```

import torch
import torch.nn as nn
from typing import NamedTuple

class MyModule(NamedTuple):
    result: List[int]

class TestNNModule(torch.nn.Module):
    def forward(self, a) -> MyModule:
        result = MyModule(result=a)
        return result

pdt_model = TestNNModule()

# Runs the pdt_model in eager model with the inputs provided and
# annotates the arguments of forward
scripted_model = torch.jit.script(pdt_model, example_inputs=
{pdt_model: [([10, 20, ], ), ], })

# Run the scripted_model with actual inputs
print(scripted_model([20]))

```

Detailed Documentation

Documentation

Scripting a function or `nn.Module` will inspect the source code, compile it as TorchScript code using the TorchScript compiler, and return a `ScriptModule` or `ScriptFunction`. TorchScript itself is a subset of the Python language, so not all features in Python work, but we provide enough functionality to compute on tensors and do control-dependent operations. For a complete guide, see the [TorchScript Language](#)

Reference.

Scripting a dictionary or list copies the data inside it into a TorchScript instance than can be subsequently passed by reference between Python and TorchScript with zero copy overhead.

and as a decorator `@torch.jit.script` for torchscript-classes and functions.

`obj` (Callable, class, or `nn.Module`) – The `nn.Module`, function, class type, dictionary, or list to compile.

`example_inputs` (Union[List[Tuple], Dict[Callable, List[Tuple]], None]) – Provide example inputs to annotate the arguments for a function or `nn.Module`.

If `obj` is `nn.Module`, `script` returns a `ScriptModule` object. The returned `ScriptModule` will have the same set of sub-modules and parameters as the original `nn.Module`. If `obj` is a standalone function, a `ScriptFunction` will be returned. If `obj` is a dict, then `script` returns an instance of `torch._C.ScriptDict`. If `obj` is a list, then `script` returns an instance of `torch._C.ScriptList`.

The `@torch.jit.script` decorator will construct a `ScriptFunction` by compiling the body of the function.

Example (scripting a function):

Example inputs can be used to annotate a function arguments.

Example (annotating a function before scripting):

Scripting an `nn.Module` by default will compile the forward method and recursively compile any methods, submodules, and functions called by forward. If a `nn.Module` only uses features supported in TorchScript, no changes to the original module code

should be necessary. script will construct ScriptModule that has copies of the attributes, parameters, and methods of the original module.

Example (scripting a simple module with a Parameter):

Example (scripting a module with traced submodules):

To compile a method other than forward (and recursively compile anything it calls), add the `@torch.jit.export` decorator to the method. To opt out of compilation use `@torch.jit.ignore` or `@torch.jit.unused`.

Example (an exported and ignored method in a module):

Example (Annotating forward of nn.Module using example_inputs):